

Simulador do Algoritmo do Banqueiro

Manual de Uso e Guia de Apresentação

Prevenção de deadlock por alocação de recursos: como o simulador funciona, como operá-lo e um roteiro pronto para apresentar em sala.

Aluno(a): _____

Disciplina: _____

Data: 28 de agosto de 2026

Sumário

- 1. Introdução
- 2. Conceitos fundamentais (deadlock e o Algoritmo do Banqueiro)
- 3. Visão geral do programa
- 4. Guia de uso passo a passo
- 5. Demonstração guiada dos três cenários prontos
- 6. Roteiro sugerido para a apresentação
- 7. Perguntas frequentes esperadas
- 8. Detalhes técnicos do projeto
- 9. Conclusão

1. Introdução

Em um sistema operacional, vários processos disputam recursos limitados (memória, arquivos, impressoras, e assim por diante). Quando um processo retém um recurso e espera por outro que está retido por um segundo processo -- que por sua vez espera por algo retido pelo primeiro -- nenhum dos dois consegue prosseguir. Esse travamento permanente é chamado de deadlock (impasse).

Este projeto implementa e visualiza o Algoritmo do Banqueiro, uma estratégia clássica (Dijkstra) para prevenir deadlocks: antes de conceder qualquer requisição de recurso, o algoritmo simula a alocação e verifica se ainda existe uma sequência em que todos os processos conseguem terminar. Se não existir, a requisição é negada.

Este documento serve para duas coisas: (1) ensinar a operar o simulador passo a passo, com telas reais do programa, e (2) fornecer um roteiro pronto -- com explicações, ordem sugerida e perguntas esperadas -- para apresentar o trabalho em sala de aula.

2. Conceitos Fundamentais

2.1 A analogia do corredor estreito

Imagine um corredor estreito da escola, largo o bastante para uma pessoa por vez. Dois grupos de amigos entram em direções opostas. Se cada grupo segurar sua posição esperando o outro "dar espaço", os dois ficam parados -- cada um esperando um recurso (o espaço livre do corredor) que só o outro pode liberar. Ninguém andou errado sozinho: o problema é a combinação das decisões de todos. É exatamente essa a situação que o Algoritmo do Banqueiro evita entre processos e recursos.

2.2 As quatro condições necessárias para deadlock

Um deadlock só ocorre se estas quatro condições estiverem presentes ao mesmo tempo:

1. Exclusão mútua -- o recurso só pode ser usado por um processo por vez.
2. Posse e espera -- um processo mantém o que já tem enquanto espera por mais.
3. Não preempção -- ninguém pode tomar à força um recurso de outro processo.
4. Espera circular -- existe uma cadeia circular de processos, cada um esperando um recurso retido pelo próximo.

O Algoritmo do Banqueiro ataca diretamente a condição de espera circular: ele nunca concede uma requisição que possa, no futuro, levar a essa cadeia fechada.

2.3 Estado seguro, estado inseguro e deadlock

- Estado seguro: existe pelo menos uma ordem em que todos os processos conseguem terminar.
- Estado inseguro: não há garantia dessa ordem -- ainda não é um deadlock, mas já é arriscado o suficiente para o Banqueiro recusar a próxima requisição perigosa.
- Deadlock: a espera circular já aconteceu de fato -- os processos envolvidos nunca mais progredem sozinhos.

2.4 As matrizes do algoritmo

- Allocation -- quanto cada processo já está segurando de cada recurso.
- Max -- o máximo que cada processo pode vir a pedir (declarado no início).

- $Need = Max - Allocation$ -- quanto cada processo ainda pode precisar.
- Available -- quanto sobrou livre de cada recurso.

A cada requisição, o simulador testa, para cada processo ainda não finalizado, se $Need \leq Work$ (o que está livre no momento do teste). Se sim, aquele processo poderia terminar e devolver tudo que tem, aumentando o Work disponível para os próximos. Se, no final da varredura, todos os processos puderem ser encaixados nessa ordem, o estado é seguro.

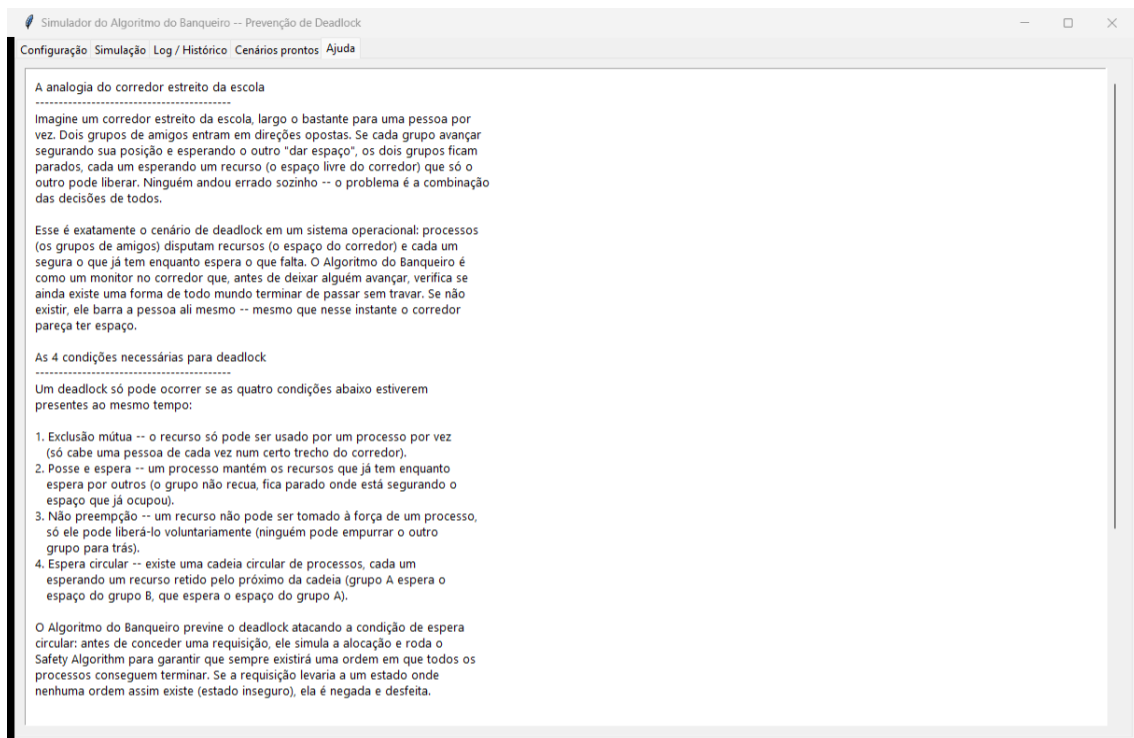


Figura 1 — Aba Ajuda do simulador, com a analogia do corredor e as 4 condições de deadlock.

3. Visão Geral do Programa

O simulador é um programa Python de janela única (Tkinter), com o grafo desenhado via matplotlib/networkx, organizado em 5 abas:

Aba	O que faz
Configuração	Cadastro manual de recursos e processos; inicia uma simulação do zero (Allocation = 0).
Simulação	Grafo de alocação de recursos, matrizes Allocation/Need/Available e formulário de requisição.
Log / Histórico	Raciocínio passo a passo do algoritmo e uma linha do tempo com snapshots do estado.
Cenários prontos	Três cenários pré-configurados para carregar com um clique (A, B e C).
Ajuda	A analogia do corredor e a explicação das condições de deadlock.

Como executar

Dentro da pasta do projeto, com Python 3.10+ instalado:

```
python main.py
```

É necessário ter as bibliotecas matplotlib e networkx instaladas (tkinter já vem com o Python no Windows). Para rodar os testes automatizados do algoritmo:

```
python -m unittest discover -s tests -v
```

4. Guia de Uso Passo a Passo

4.1 Aba Configuração -- montando uma simulação do zero

A aba começa vazia, com um formulário para recursos e outro para processos:

Simulador do Algoritmo do Banqueiro -- Prevenção de Deadlock

Configuração Simulação Log / Histórico Cenários prontos Ajuda

Recursos

Nome: Instâncias totais: 1

Nome	Total de instâncias
------	---------------------

Processos

Nome: (cadastre ao menos um recurso antes de adicionar processos)

Nome	Max (uma coluna por recurso)
------	------------------------------

Todos os processos começam sem nenhum recurso alocado; use a aba Simulação para fazer requisições.

Figura 2 — Aba Configuração, ainda vazia.

1. Cadastre um recurso: digite um nome e a quantidade total de instâncias, e clique em "Adicionar Recurso".

Simulador do Algoritmo do Banqueiro -- Prevenção de Deadlock

Configuração Simulação Log / Histórico Cenários prontos Ajuda

Recursos

Nome: Instâncias totais: 3

Nome	Total de instâncias
A	5
B	3

Processos

Nome: Max A: 0 Max B: 0

Nome	Max (uma coluna por recurso)
------	------------------------------

Todos os processos começam sem nenhum recurso alocado; use a aba Simulação para fazer requisições.

Figura 3 — Dois recursos (A e B) já cadastrados.

2. Cadastre um processo: digite um nome e, para cada recurso já cadastrado, o máximo que esse processo pode vir a pedir. Clique em "Adicionar Processo".

Simulador do Algoritmo do Banqueiro -- Prevenção de Deadlock

Configuração Simulação Log / Histórico Cenários prontos Ajuda

Recursos

Nome: Instâncias totais: 3

Nome	Total de instâncias
A	5
B	3

Processos

Nome: Max A: 0 Max B: 0

Nome	Max (uma coluna por recurso)
P0	[3, 2]
P1	[2, 3]

Todos os processos começam sem nenhum recurso alocado; use a aba Simulação para fazer requisições.

Figura 4 — Dois processos (P0 e P1) já cadastrados, cada um com seu vetor Max.

3. Clique em "Iniciar Simulação (Allocation = 0)". O programa monta o estado inicial -- todos os processos começam sem nenhum recurso alocado -- e leva você para a aba Simulação.

Simulador do Algoritmo do Banqueiro -- Prevenção de Deadlock

Configuração Simulação Log / Histórico Cenários prontos Ajuda

Grafo de Alocação de Recursos

Modo Protegido -- P0: pronto | P1: pronto

Allocation

	A	B
P0	0	0
P1	0	0

Need (Max - Allocation)

	A	B
P0	3	2
P1	2	3

Available

	A	B
livres	5	3

Fazer Requisição / Ação

Processo:

A: B:

☒ Protegido (com Banqueiro)
☐ Livre (sem prevenção)

Figura 5 — Simulação recém-iniciada: Allocation zerada, grafo sem nenhuma seta ainda.

4.2 Aba Simulação -- o grafo, as matrizes e o formulário de requisição

É a tela principal, dividida em duas partes:

- À esquerda: o Grafo de Alocação de Recursos. Círculos são processos, quadrados são recursos (com "livre/total" escrito dentro). Setas pretas cheias indicam alocação (recurso -> processo); setas vermelhas tracejadas indicam um pedido pendente ou negado (processo -> recurso).
- À direita: as tabelas Allocation, Need e Available, sempre atualizadas, e o formulário para escolher um processo, digitar quanto de cada recurso ele está pedindo, e clicar em "Solicitar".

As cores dos processos também têm significado: verde é pronto/executando, amarelo é bloqueado (esperando um recurso) e cinza é terminado.

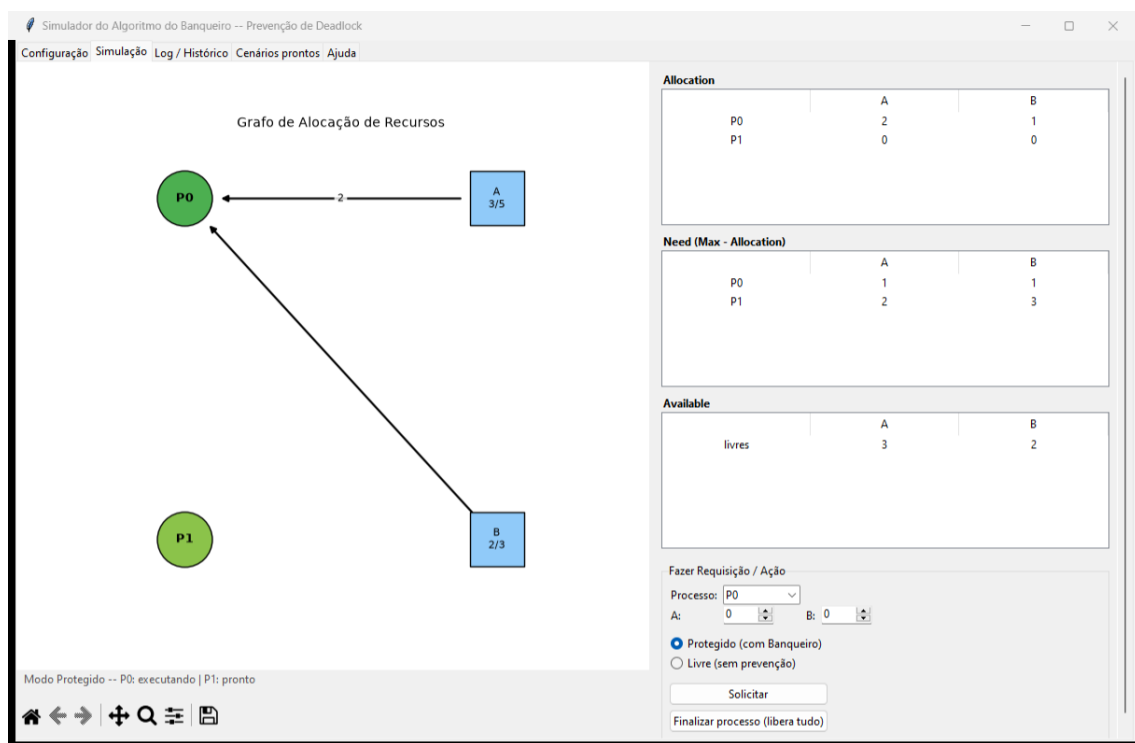


Figura 6 — Depois de um pedido concedido: a Allocation do processo escolhido aumentou e o Available diminuiu.

Repare também nos dois botões de rádio "Protegido (com Banqueiro)" e "Livre (sem prevenção)": eles alternam se o simulador vai verificar segurança antes de conceder cada pedido ou não -- isso é essencial para o Cenário C, explicado adiante.

4.3 Aba Log / Histórico -- o raciocínio do algoritmo

Toda vez que algo acontece (um pedido, uma negação, uma detecção de deadlock), o programa registra cada etapa do raciocínio nessa aba: quais comparações $Need \leq Work$ foram feitas, se cada processo pôde prosseguir, e a conclusão final. Do lado direito fica a linha do tempo (timeline), com um resumo de cada evento -- clicando em um deles, é possível ver como as matrizes estavam naquele momento.

4.4 Aba Cenários prontos

A forma mais rápida de demonstrar o simulador: três botões, cada um carregando uma situação já configurada (explicados em detalhe na seção 5).

4.5 Aba Ajuda

Contém a analogia do corredor e a explicação das quatro condições de deadlock, útil para consultar durante a apresentação (ver Figura 1).

5. Demonstração Guiada dos Três Cenários

Esta é a parte mais importante para a apresentação: os três cenários prontos mostram, na prática, um estado seguro, uma negação e um deadlock real.

5.1 Cenário A -- Estado seguro (exemplo clássico)

5 processos disputando 3 tipos de recursos (A=10, B=5, C=7). Ao carregar, o estado já é seguro: existe uma sequência de execução -- <P1, P3, P4, P0, P2> -- em que todos os processos conseguem terminar sem travar. Esse é o exemplo clássico do livro de Sistemas Operacionais (Silberschatz).

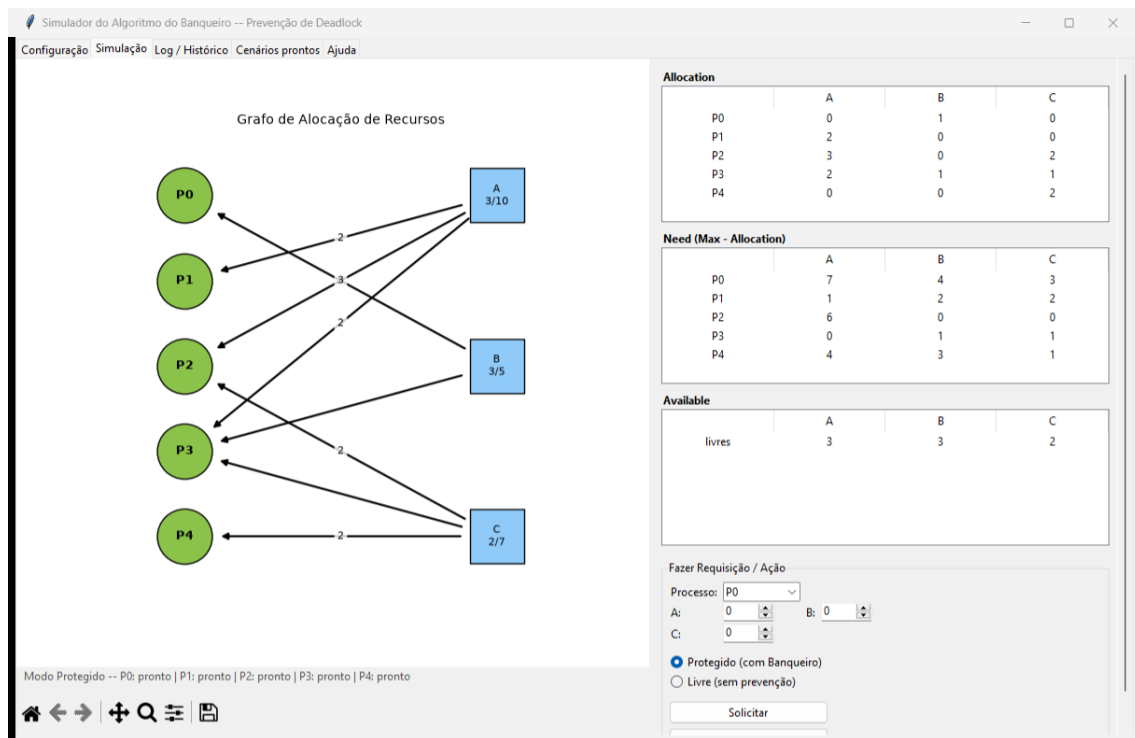


Figura 7 — Cenário A carregado: grafo, matrizes Allocation/Need/Available e status "pronto" para todos os processos.

O que dizer na apresentação: mesmo com vários processos precisando de bastante recurso (veja a coluna Need), sempre existe alguém que pode terminar primeiro e devolver o que usou, destravando os próximos -- por isso o estado é seguro.

5.2 Cenário B -- Requisição negada (estado inseguro)

3 processos disputando 2 recursos (A=3, B=3). O estado inicial é seguro, mas a descrição do cenário sugere um pedido perigoso: o processo P1 solicitando (1, 0).

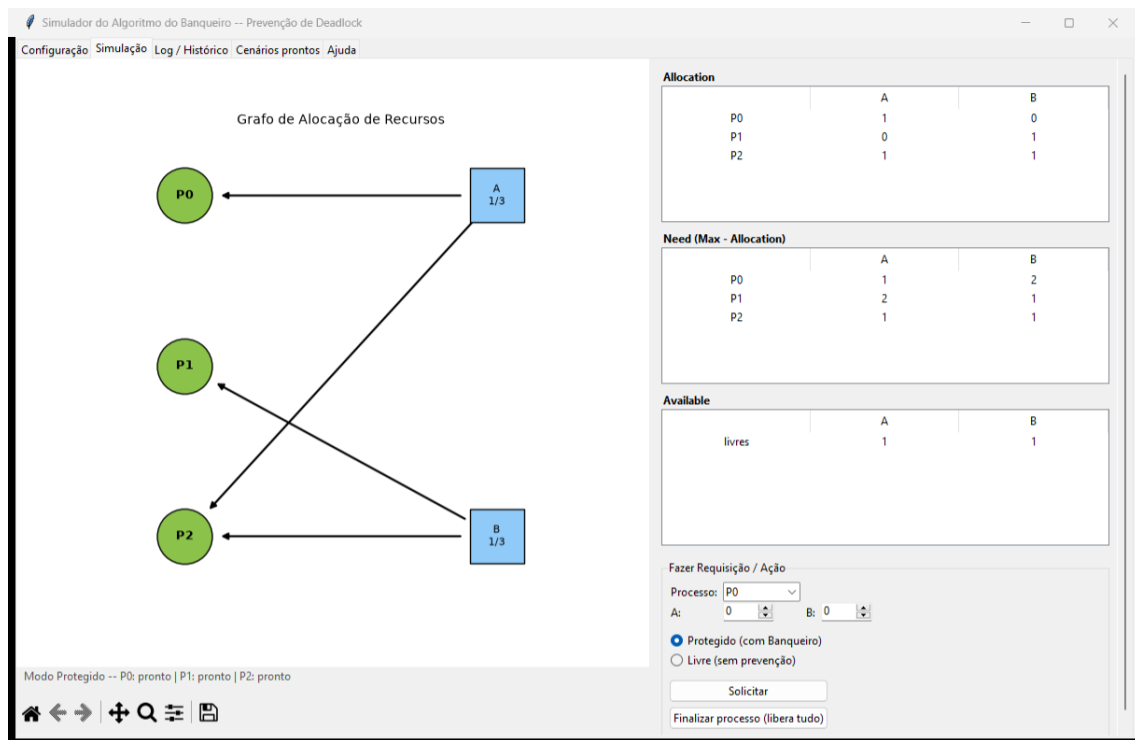


Figura 8 — Cenário B logo após carregar -- ainda em estado seguro.

Ao ir na aba Simulação, escolher P1, digitar 1 no primeiro recurso e 0 no segundo, e clicar em Solicitar, o pedido é negado:

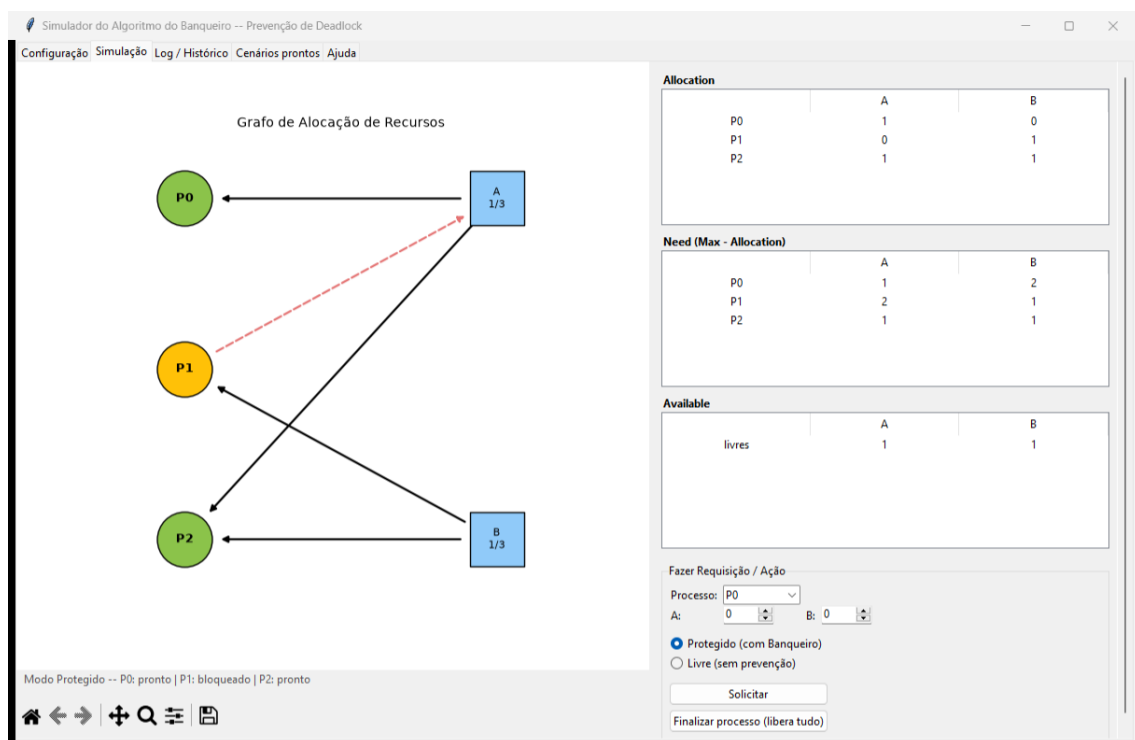


Figura 9 — Pedido de P1 negado: a Allocation voltou exatamente ao que era antes (rollback), P1 aparece bloqueado (amarelo) com uma seta vermelha tracejada pendente.

Na aba Log, é possível mostrar o motivo exato:

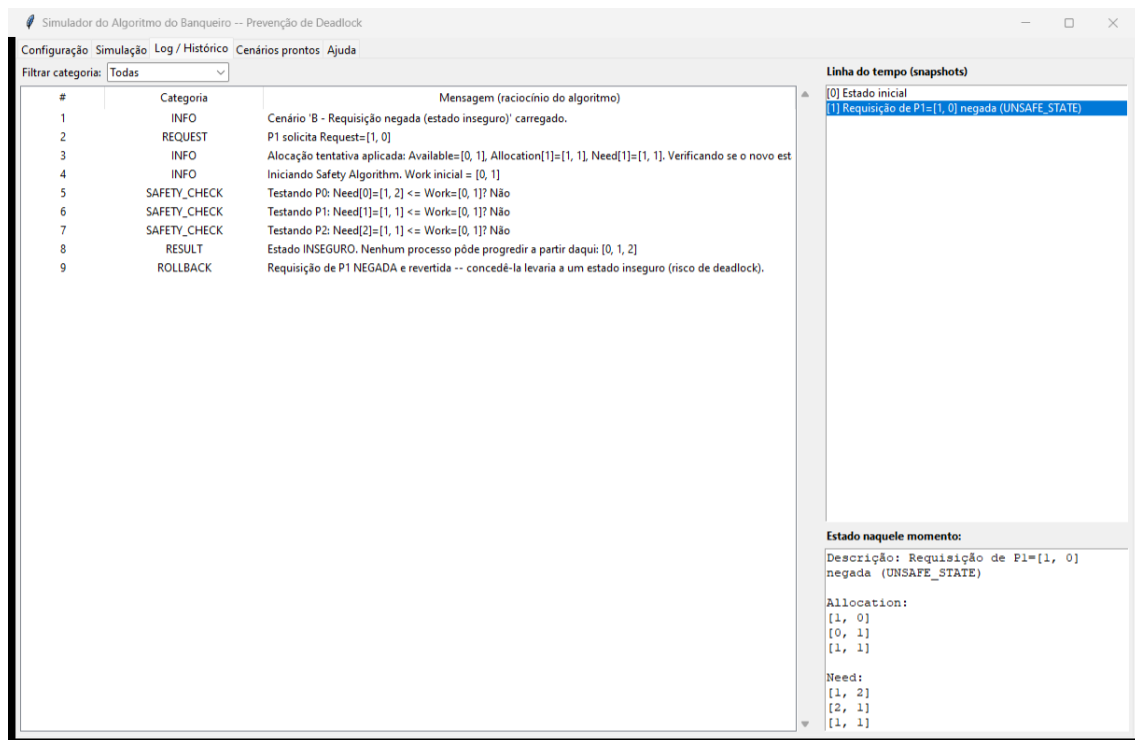


Figura 10 — Log do Cenário B: cada comparação $Need \leq Work$ aparece, terminando em "Estado INSEGURO" e "ROLLBACK".

O que dizer na apresentação: mesmo havendo recurso disponível na hora do pedido, o Banqueiro enxerga à frente e percebe que, se conceder, ninguém mais conseguiria progredir depois -- por isso prefere negar e desfazer a alocação (rollback) a arriscar um travamento.

5.3 Cenário C -- Deadlock real por espera circular (modo Livre)

3 processos e 3 recursos de instância única, cada um já alocado por um processo diferente. Esse cenário carrega automaticamente em modo Livre -- ou seja, sem a proteção do Banqueiro -- exatamente para mostrar o que aconteceria sem o algoritmo.

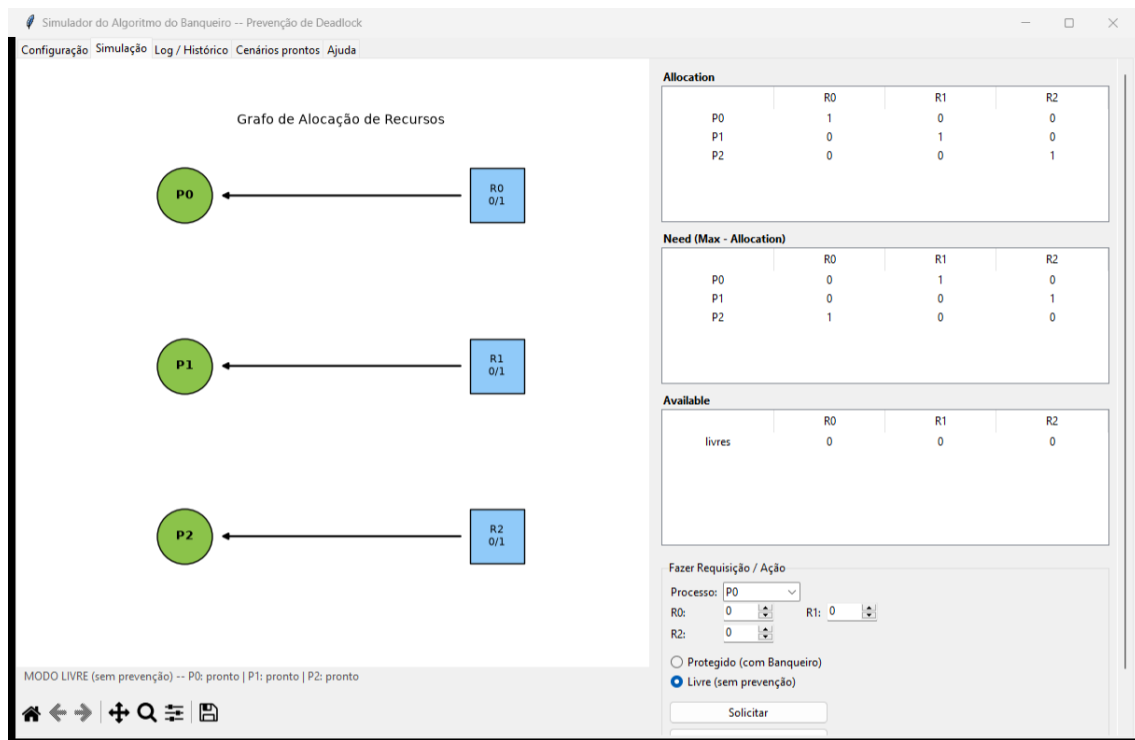


Figura 11 — Cenário C recém-carregado: cada processo já segura um recurso, Available = 0 em tudo.

Seguindo os pedidos sugeridos na descrição do cenário (P0 pede o recurso de P1, P1 pede o de P2, e P2 pede o de P0), as três requisições ficam pendentes e fecham um ciclo:

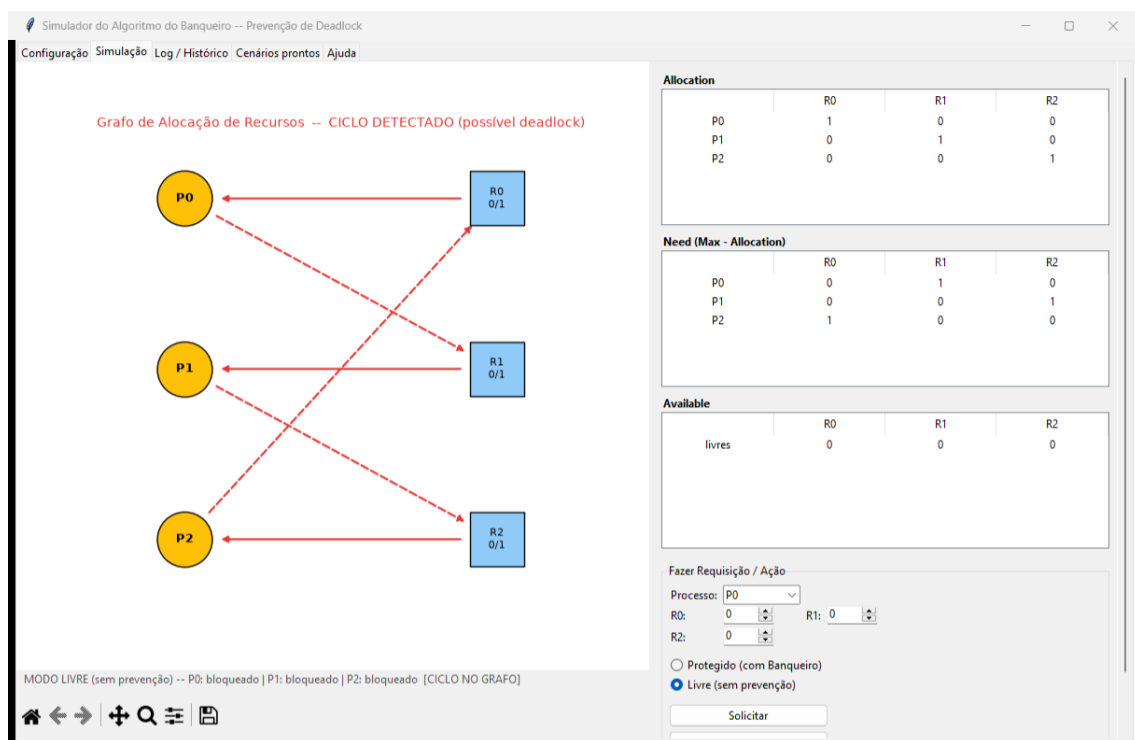


Figura 12 — As três requisições fecham um ciclo -- destacado em vermelho no grafo -- e o título já avisa "CICLO DETECTADO".

Clicando em "Verificar Deadlock", o algoritmo de detecção confirma formalmente o travamento:

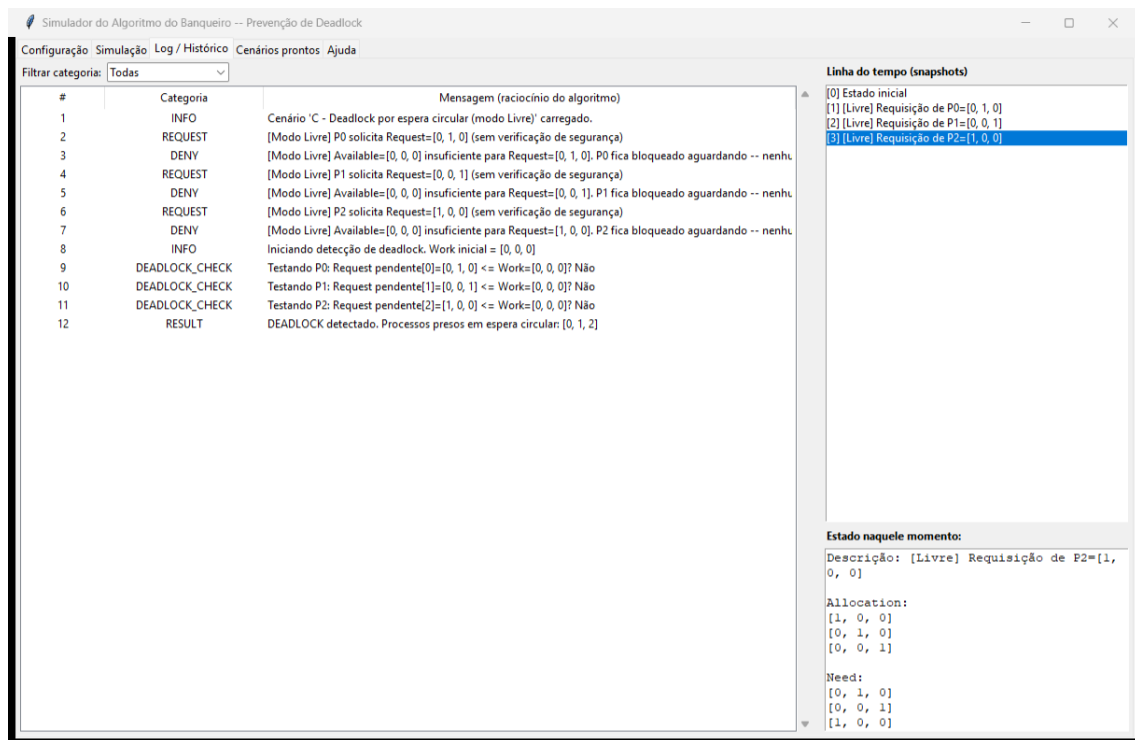


Figura 13 — Log de detecção: cada processo é testado contra o que está pendente, e nenhum consegue progredir -- "DEADLOCK detectado".

O que dizer na apresentação: esse é o contraste-chave do trabalho -- se esse mesmo cenário rodasse em modo Protegido, a segunda ou terceira requisição já teria sido negada antes de o ciclo se fechar, exatamente como no Cenário B.

6. Roteiro Sugerido para a Apresentação

Um roteiro de aproximadamente 8 a 10 minutos, usando só a aba Cenários prontos e a de Log:

Tempo	O que fazer	O que falar
~1 min	Abrir na aba Ajuda	Contar a analogia do corredor e citar as 4 condições de deadlock.
~2 min	Carregar Cenário A	Explicar Allocation/Max/Need/Available e por que esse estado é seguro.
~2 min	Carregar Cenário B e fazer o pedido de P1	Mostrar a negação no grafo e depois o raciocínio linha a linha no Log.
~3 min	Carregar Cenário C, fazer os 3 pedidos e clicar em Verificar Deadlock	Mostrar o ciclo em vermelho e a confirmação no Log; comparar com o que aconteceria em modo Protegido.
~1 min	Encerramento	Resumir: o Banqueiro nunca deixa o sistema chegar a um deadlock, recusando qualquer pedido que levaria a um estado inseguro.

Dica: pratique a sequência de cliques antes da apresentação (Cenários prontos -> Simulação -> Log) para não perder tempo procurando os botões na hora.

7. Perguntas Frequentes Esperadas

P: Por que o Banqueiro nega um pedido se, naquele instante, existe recurso disponível?

R: Porque "ter disponível agora" não é suficiente -- o algoritmo simula a concessão e verifica se, depois dela, ainda existe alguma ordem em que todos os processos conseguem terminar. Se não existir, é arriscado demais conceder.

P: Qual a diferença entre estado inseguro e deadlock?

R: Estado inseguro é quando não há garantia de uma sequência segura -- pode ou não terminar em deadlock, dependendo do que cada processo pedir depois. Deadlock é quando a espera circular já aconteceu de fato e ninguém mais progride sozinho.

P: Como o simulador consegue mostrar um deadlock de verdade, se o Banqueiro sempre previne?

R: Existe um modo "Livre", usado no Cenário C, que desliga a verificação de segurança de propósito, só para fins didáticos -- assim é possível ver a espera circular se formando e sendo confirmada por um algoritmo de detecção separado.

P: O que acontece quando uma requisição é negada -- os recursos ficam perdidos?

R: Não. O simulador testa a alocação, e se o resultado for inseguro, desfaz exatamente essa tentativa (rollback) -- as matrizes voltam a ser idênticas ao estado anterior ao pedido.

P: Esse algoritmo é usado em sistemas operacionais reais?

R: O Algoritmo do Banqueiro é mais um modelo teórico/didático (proposto por Edsger Dijkstra) do que uma solução usada diretamente em SOs de propósito geral, porque exige saber de antemão o máximo que cada processo vai precisar -- algo raramente conhecido na prática. Sistemas reais tendem a usar outras estratégias, como detecção e recuperação, ou simplesmente evitar ciclos de espera por convenção (ex.: sempre pedir recursos em uma mesma ordem).

P: Por que as matrizes usam o vetor Need em vez de Max diretamente?

R: $\text{Need} = \text{Max} - \text{Allocation}$ representa o que falta, que é exatamente o que interessa para saber se um processo consegue terminar com o que ainda está disponível (Work).

8. Detalhes Técnicos do Projeto

Para quem quiser (ou for perguntado sobre) a implementação:

- Linguagem: Python 3, interface gráfica com Tkinter.
- Grafo de alocação: matplotlib + networkx, embutido na janela.
- core/ -- lógica pura do algoritmo (Safety Algorithm, Resource-Request Algorithm, detecção de deadlock), sem nenhuma dependência de interface gráfica.
- simulation/ -- motor de simulação: modos Protegido e Livre, histórico de eventos e snapshots.
- scenarios/ -- definição dos três cenários prontos (A, B e C).
- gui/ -- as cinco abas da interface.
- tests/ -- 33 testes automatizados cobrindo o algoritmo (incluindo o caso clássico do livro-texto e o teste de rollback exato).

```
python -m unittest discover -s tests -v
```

roda toda a suíte de testes e mostra cada verificação passando.

9. Conclusão

O simulador cobre, de ponta a ponta, o ciclo pedido pelo trabalho: simular múltiplos processos disputando recursos, implementar o Algoritmo do Banqueiro completo (incluindo o Safety Algorithm), visualizar o grafo de alocação, representar as matrizes de estado, demonstrar cenários que levam a deadlock e como são prevenidos, e registrar em log o raciocínio de cada decisão.

Para a apresentação, o caminho mais direto é: Ajuda -> Cenário A -> Cenário B (com o pedido negado) -> Cenário C (com o deadlock real) -> Log. Esse percurso, sozinho, já conta toda a história do algoritmo.